```
"John Smith".parameterize # => "john-smith"
"Kurt Gödel".parameterize # => "kurt-godel"
```

In fact, the result string is wrapped in an instance of `ActiveSupport::Multibyte::Chars`.

Defined in `active_support/core_ext/string/inflections.rb`.

5.11.10 `tableize`

The method `tableize` is `underscore` followed by `pluralize`.

```
"Person".tableize      # => "people"
"Invoice".tableize     # => "invoices"
"InvoiceLine".tableize # => "invoice_lines"
```

As a rule of thumb, `tableize` returns the table name that corresponds to a given model for simple cases. The actual implementation in Active Record is not straight `tableize` indeed, because it also demodulizes the class name and checks a few options that may affect the returned string.

Defined in `active_support/core_ext/string/inflections.rb`.

5.11.11 `classify`

The method `classify` is the inverse of `tableize`. It gives you the class name corresponding to a table name:

```
"people".classify        # => "Person"
"invoices".classify      # => "Invoice"
"invoice_lines".classify # => "InvoiceLine"
```

The method understands qualified table names:

```
"highrise_production.companies".classify # => "Company"
```

Note that `classify` returns a class name as a string. You can get the actual class object invoking `constantize` on it, explained next.

Defined in `active_support/core_ext/string/inflections.rb`.

5.11.12 `constantize`

The method `constantize` resolves the constant reference expression in its receiver:

```
"Fixnum".constantize # => Fixnum

module M
  X = 1
end
"M::X".constantize # => 1
```

If the string evaluates to no known constant, or its content is not even a valid constant name, `constantize` raises `NameError`.

Constant name resolution by `constantize` starts always at the top-level `Object` even if there is no leading "::".

```
X = :in_Object
module M
  X = :in_M

  X         # => :in_M
  "::X".constantize # => :in_Object
  "X".constantize   # => :in_Object (!)
end
```

So, it is in general not equivalent to what Ruby would do in the same spot, had a real constant be evaluated.

Mailer test cases obtain the mailer being tested from the name of the test class using `constantize`:

```
# action_mailer/test_case.rb
def determine_default_mailer(name)
  name.sub(/Test$/, '').constantize
rescue NameError => e
  raise NonInferrableMailerError.new(name)
```